

MINISTÉRIO DA DEFESA
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
(Real Academia de Artilharia, Fortificação e Desenho – 1792)

SEÇÃO DE ENSINO (SE/3) - ENGENHARIA DE COMUNICAÇÕES
ARQUITETURA DE COMPUTADORES

TRADUÇÃO DE ALGORITMOS DE MIPS ASM E LINGUAGEM C
PARA EXCEL-16 ASM

ALUNO:
LUCAS MENDES SANTIAGO FILHO

Sumário

1	Objetivo	3
2	Ferramentas Utilizadas	3
3	Arquitetura do ExcelCPU (Excel-ASM16)	3
3.1	Instruções Utilizadas Neste Trabalho	4
4	Exemplo 1 — Área de um Retângulo	4
4.1	O Algoritmo Original e sua Verificação no JsSpim	4
4.2	Código MIPS Original	5
4.3	Decisões de Tradução	5
4.4	Desafio Encontrado: Gravação do Resultado	5
4.5	Mapeamento de Registradores e Memória	6
4.6	Código Final Traduzido	6
4.7	Resultado	7
5	Exemplo 2 — Conversão RGB para Tons de Cinza	7
5.1	O Algoritmo Original (C)	7
5.2	Decisões de Tradução	8
5.3	Mapeamento de Registradores e Layout de Memória	8
5.4	Código Final Traduzido	8
5.5	Resultado	10
6	Compilação e Execução	11
7	Conclusão	11

1. Objetivo

O objetivo deste trabalho é traduzir dois algoritmos para a linguagem de montagem **Excel-ASM16**, utilizada pela CPU de 16 bits implementada inteiramente em planilha Excel (*ExcelCPU*):

1. **Exemplo 1** (`ex1.asm`) — programa em MIPS Assembly que lê a largura e a altura de um retângulo, calcula sua área e imprime o resultado;
2. **Exemplo 2** (`ex2.c`) — programa em linguagem C que converte um conjunto de pixels RGB (*Red-Green-Blue*) em valores de tom de cinza.

Assim como no trabalho anterior, não se trata de criar novas implementações, mas de realizar uma **tradução fiel** da lógica existente, respeitando o fluxo de cada programa original e adaptando-o às restrições da arquitetura de destino.

2. Ferramentas Utilizadas

Ferramenta	Descrição
JsSpim	Simulador online MIPS32 — utilizado para verificar o Exemplo 1 antes da tradução
GCC	Compilador C — utilizado para compilar e executar o Exemplo 2 original e obter as saídas de referência
ExcelCPU	CPU de 16 bits implementada em Excel; inclui <code>CPU.xlsx</code> , <code>ROM.xlsx</code> e o compilador Python
<code>compileExcelASM16.py</code>	Script Python que compila o arquivo <code>.s</code> e grava o programa no <code>ROM.xlsx</code>

3. Arquitetura do ExcelCPU (Excel-ASM16)

O ExcelCPU é uma CPU de **16 bits** implementada inteiramente em fórmulas do Microsoft Excel, com:

- 16 registradores de uso geral (R0–R15);
- RAM endereçada por 16 bits (cada posição armazena uma palavra de 16 bits);
- display de 128×128 pixels, mapeado em memória a partir do endereço `$F000`;
- conjunto de instruções **Excel-ASM16** (24 instruções) e dois *flags*: *carry* (CF) e *zero* (ZF).

Uma característica importante da arquitetura é que ela **não possui dispositivo de entrada por hardware** (não há teclado nem `stdin`). A entrada de dados é feita editando a planilha `ROM.xlsx` antes da execução; a saída é observada lendo as células de RAM da `CPU.xlsx` após a execução, ou desenhada no display.

3.1. Instruções Utilizadas Neste Trabalho

Instrução	Exemplo	Descrição
LOAD REG IMD	LOAD R1 #4	Carrega valor imediato (decimal ou hex) no registrador
LOAD REG MEM	LOAD R1 WIDTH	Carrega o conteúdo de um endereço fixo da RAM
LOAD REG REG	LOAD R4 R3	Carrega da RAM no endereço contido no 2º registrador (indireto)
STORE REG REG	STORE R1 R0	Escreve no endereço contido no 2º registrador
TRAN REG REG	TRAN R5 R6	Copia o valor de um registrador para outro
INC REG	INC R1	Incrementa em 1 (não afeta CF)
MULT REG REG	MULT R1 R2	Multiplica; parte baixa do produto em REGA, parte alta em REGB
DIV REG REG	DIV R4 R0	Divide REGA por REGB; quociente em REGA, resto em REGB
ADD REG REG	ADD R4 R6	Soma: REGA + REGB + CF, resultado em REGA
AND REG REG	AND R4 R8	E lógico bit a bit, resultado em REGA
ROR REG IMD	ROR R6 #8	Rotação de bits à direita, IMD vezes
CMP REG REG	CMP R4 R7	Compara (subtrai) sem guardar o resultado; só afeta os flags
CLC	CLC	Zera o <i>carry flag</i> (CF = 0)
JEQ label	JEQ DONE	Salta se ZF = 1
JMP label	JMP LOOP	Salto incondicional

Observação geral sobre o simulador. Durante o desenvolvimento foram identificados dois comportamentos da CPU que afetam diretamente a tradução e que valem para ambos os exercícios:

- As instruções aritméticas ADD e SUB **incluem o carry flag** na operação (REGA + REGB + CF). Por isso é necessário executar CLC antes de uma soma “limpa”.
- Carregar um valor imediato (LOAD Rn #x) em um **registrador alto** (R8–R15) e utilizá-lo *imediatamente* na instrução seguinte (como ponteiro de STORE ou operando de DIV) não funciona de forma confiável: o valor recém-carregado ainda não se propagou. O uso de **registradores baixos** (R0–R7) nesses casos resolve o problema.

4. Exemplo 1 — Área de um Retângulo

4.1. O Algoritmo Original e sua Verificação no JsSpim

O programa MIPS lê dois inteiros da entrada padrão (largura e altura), calcula a área como o produto dos dois e imprime o resultado. Antes da tradução, o arquivo `ex1.asm` foi carregado no simulador JsSpim para confirmar o comportamento esperado:

Entrada: largura = 5, altura = 3

Saída: Rectangle's area is 15

4.2. Código MIPS Original

Listing 1: ex1.asm (trecho principal)

```
1 main:
2     addi $v0, $0, 4          # syscall 4: imprime string (prompt da
3                               largura)
4     la    $a0, widthPrompt
5     syscall
6     addi $v0, $0, 5          # syscall 5: le inteiro
7     syscall
8     add   $8, $0, $v0        # $8 = largura
9
10    addi $v0, $0, 4          # prompt da altura
11    la    $a0, heightPrompt
12    syscall
13    addi $v0, $0, 5          # le inteiro
14    syscall
15    add   $9, $0, $v0        # $9 = altura
16
17    mult  $8, $9              # multiplica largura * altura
18    mflo  $10                 # $10 = produto
19
20    addi $v0, $0, 4          # imprime "Rectangle's area is "
21    la    $a0, areaIs
22    syscall
23    addi $v0, $0, 1          # syscall 1: imprime inteiro
24    add   $a0, $0, $10
25    syscall
```

4.3. Decisões de Tradução

- **Entrada de dados.** Como a Excel-16 não possui `stdin`, as *syscalls* de leitura foram substituídas pela leitura de duas posições fixas da memória (\$0002 e \$0003). Os valores de largura e altura são fornecidos pelo usuário editando a `ROM.xlsx` antes da execução — o equivalente, nesta arquitetura, à “entrada do usuário”.
- **Saída de dados.** As *syscalls* de impressão de texto foram removidas (a CPU não imprime strings). O resultado é gravado na memória, no endereço \$0004, e lido diretamente na planilha após a execução.
- **Multiplicação.** O MIPS usa `mult` seguido de `mflo` para obter o produto. Na Excel-16, a instrução `MULT` já entrega o resultado de 32 bits diretamente em dois registradores: a parte baixa em `REGA` e a parte alta em `REGB`. Por isso, foram reservadas duas posições de saída (\$0004 para a parte baixa e \$0005 para a parte alta), cobrindo o caso em que o produto ultrapassa 16 bits.

4.4. Desafio Encontrado: Gravação do Resultado

A primeira versão usava a instrução `STORE` com endereço absoluto (`STORE R1 @0004`). Após a execução, a posição de memória permanecia zerada, mesmo com o programa

atingindo o fim corretamente. Investigando, constatou-se que:

1. A variante `STORE REG MEM` (endereço absoluto) não atualizava a RAM de forma confiável neste simulador. A solução foi usar `STORE REG REG`, em que o endereço-destino fica em um registrador — a mesma forma usada nos programas de exemplo do ExcelCPU.
2. Mesmo após essa correção, o resultado só passou a aparecer quando o ponteiro foi colocado em um **registrador baixo**. Carregar o endereço com `LOAD R10 #4` (registrador alto) e usá-lo no `STORE` seguinte não funcionava: o valor não se propagava a tempo. Passando a usar `R0` como ponteiro, a gravação passou a ocorrer corretamente.

4.5. Mapeamento de Registradores e Memória

MIPS	Excel-ASM16	Papel
\$8	R1	Largura (lida de \$0002)
\$9	R2	Altura (lida de \$0003); recebe a parte alta do produto
\$10	R1	Parte baixa do produto (área)
—	R0	Ponteiro de gravação na memória

Endereço	Variável	Conteúdo
\$0002	WIDTH	largura (entrada)
\$0003	HEIGHT	altura (entrada)
\$0004	AREA	área — parte baixa (saída)
\$0005	AREAHI	área — parte alta (saída)

4.6. Código Final Traduzido

Listing 2: ex1.s — versão final em Excel-ASM16

```

1  .DATA
2  WIDTH = #0      ; $0002 - entrada
3  HEIGHT = #0     ; $0003 - entrada
4  AREA = #0       ; $0004 - saída (parte baixa)
5  AREAHI = #0     ; $0005 - saída (parte alta, se passar de 65535)
6
7  .CODE
8  LOAD R1 WIDTH
9  LOAD R2 HEIGHT
10
11 ; area = largura * altura
12 CLC          ; zera o carry antes (ADD/MULT levam o carry em
13              conta)
14 MULT R1 R2    ; R1 = parte baixa, R2 = parte alta (R2 e
15              sobrescrito)
16
17 ; grava o resultado (R0 e baixo de proposito - ver secao de
18              desafios)

```

```
16 LOAD R0 #4
17 STORE R1 R0      ; $0004 = area
18 LOAD R0 #5
19 STORE R2 R0      ; $0005 = parte alta
20
21 ; nao existe HALT, entao travo num loop pra terminar
22 END:
23 JMP END
```

O programa compila em **20 palavras**. Após a execução, o contador de programa (PC) fica travado no endereço 17 (END).

4.7. Resultado

Entrada (na ROM): largura = 5, altura = 3

Saída (em \$0004): 15

O resultado coincide com a saída do simulador MIPS. O programa também foi verificado com outros valores — por exemplo, $10 \times 10 = 100$ — e, no caso de produtos maiores que 16 bits (como $1000 \times 1000 = 1\,000\,000$), as partes baixa (\$0004) e alta (\$0005) juntas reproduzem o valor de 32 bits, exatamente como o par `mflo/mfhi` faria no MIPS.

5. Exemplo 2 — Conversão RGB para Tons de Cinza

5.1. O Algoritmo Original (C)

O programa em C percorre um vetor de pixels de 24 bits (formato `0xRRGGBB`) até encontrar o valor sentinela `-1`. Para cada pixel, extrai os canais vermelho, verde e azul por meio de deslocamentos e máscaras, calcula a média $(R + G + B) / 3$ e imprime o tom de cinza resultante.

Listing 3: ex2.c

```
1 int pixels[] = {0x00010000, 0x010101, 0x6, 0x3333,
2                 0x030c, 0x700853, 0x294999, -1};
3
4 int rgb_to_gray(int red, int green, int blue) {
5     return (red + green + blue) / 3;
6 }
7
8 int main() {
9     int i = 0, rgb, red, green, blue, gray;
10    printf("Converting pixels to grayscale:\n");
11    while (pixels[i] != -1) {
12        rgb = pixels[i];
13        blue = rgb & 0xff;
14        green = (rgb >> 8) & 0xff;
15        red = (rgb >> 16) & 0xff;
16        gray = rgb_to_gray(red, green, blue);
```

```

17     printf("%d\n", gray);
18     i++;
19 }
20 printf("Finished.\n");
21 }

```

Compilado e executado em um PC, o programa produz a sequência de tons de cinza usada como referência: 0, 1, 2, 34, 5, 67, 89.

5.2. Decisões de Tradução

- **Pixels de 24 bits em uma CPU de 16 bits.** Cada pixel 0xRRGGBB não cabe em uma única palavra de 16 bits. A solução foi armazenar cada pixel em **duas palavras consecutivas**: a primeira contém apenas o vermelho (\$00RR) e a segunda contém o verde e o azul juntos (\$GGBB). É o equivalente a separar o número em parte alta e parte baixa.
- **Sentinela.** O valor -1 que encerra o laço em C foi representado por \$FFFF na palavra alta do “pixel” final. Como um pixel válido nunca ultrapassa \$00FF na palavra alta, não há ambiguidade.
- **Extração dos canais.** A Excel-16 não possui deslocamento lógico ($\gg$). Para obter o verde $((\text{rgb} \gg 8) \& 0\text{xff})$, foi usada a instrução ROR (rotação à direita de 8 bits) seguida de AND com a máscara \$00FF, que limpa os bits que retornaram pelo topo na rotação.
- **Soma e divisão.** A soma $R + G + B$ usa ADD (com CLC antes, como discutido na Seção 3); a média usa a instrução DIV. Como DIV deixa o resto no segundo operando, o divisor (3) é recarregado a cada iteração. Seguindo a mesma lição do Exemplo 1, o divisor foi colocado em R0 (registrador baixo) para que o valor esteja disponível na DIV seguinte.
- **Saída.** O printf foi substituído pela gravação dos sete tons de cinza em posições consecutivas da RAM (a partir de \$0100) e, adicionalmente, no display (a partir de \$F000), produzindo uma saída visual.

5.3. Mapeamento de Registradores e Layout de Memória

5.4. Código Final Traduzido

Listing 4: ex2.s — versão final em Excel-ASM16

```

1  .DATA
2  ; cada pixel = 2 palavras (parte alta = R, parte baixa = GB)
3  POHI = $0001      ; 0x00010000  R=1  G=0  B=0
4  POLO = $0000
5  P1HI = $0001      ; 0x00010101  R=1  G=1  B=1
6  P1LO = $0101
7  P2HI = $0000      ; 0x00000006  R=0  G=0  B=6
8  P2LO = $0006
9  P3HI = $0000      ; 0x00003333  R=0  G=51 B=51

```


Registrador	Papel
R0	Divisor (valor 3), recarregado a cada iteração
R1	Ponteiro de leitura no vetor de pixels
R2	Ponteiro de escrita dos resultados na RAM (\$0100+)
R3	Ponteiro de escrita no display (\$F000+)
R4	Vermelho → soma → tom de cinza
R5	Azul
R6	Verde
R7	Sentinela (\$FFFF), para comparação
R8	Máscara de um byte (\$00FF)

Pixel	0xRRGGBB	R	G	B
0	0x00010000	1	0	0
1	0x00010101	1	1	1
2	0x00000006	0	0	6
3	0x00003333	0	51	51
4	0x0000030C	0	3	12
5	0x00700853	112	8	83
6	0x00294999	41	73	153

```

10 P3LO = $3333
11 P4HI = $0000      ; 0x0000030C  R=0  G=3  B=12
12 P4LO = $030C
13 P5HI = $0070      ; 0x00700853  R=112 G=8  B=83
14 P5LO = $0853
15 P6HI = $0029      ; 0x00294999  R=41  G=73 B=153
16 P6LO = $4999
17 SENT = $FFFF      ; marca o fim do array (o -1 do C)
18
19 .CODE
20 ; ponteiros e constantes
21 LOAD R1 $0002      ; comeco do array
22 LOAD R2 $0100      ; onde vou gravar os resultados
23 LOAD R3 $F000      ; display
24 LOAD R7 $FFFF      ; sentinela pra comparar
25 LOAD R8 $00FF      ; mascara de 1 byte
26
27 LOOP:
28 LOAD R4 R1          ; le a parte alta do pixel
29 CMP R4 R7           ; chegou no $FFFF?
30 JEQ DONE           ; entao acabou
31
32 INC R1
33 LOAD R5 R1          ; le a parte baixa ($GGBB)
34 INC R1              ; ja deixa apontando pro proximo pixel
35
36 ; separa R, G, B
37 AND R4 R8           ; R = parte alta & 0xFF

```

```

38
39 TRAN R5 R6          ; copia $GGBB
40 ROR R6 #8           ; gira 8 bits pra trocar os bytes
41 AND R6 R8           ; G = byte que sobrou
42
43 AND R5 R8           ; B = parte baixa & 0xFF
44
45 ; soma = R + G + B (no maximo 765, cabe em 16 bits)
46 CLC                ; ADD soma o carry junto, entao zero ele antes
47 ADD R4 R6
48 CLC
49 ADD R4 R5
50
51 ; gray = soma / 3 (divisor em R0, registrador baixo)
52 LOAD R0 #3
53 DIV R4 R0          ; R4 = soma/3 (o resto vai pro R0 e eu ignoro)
54
55 ; grava o cinza na RAM e no display
56 STORE R4 R2
57 INC R2
58 STORE R4 R3
59 INC R3
60
61 JMP LOOP
62
63 DONE:
64 JMP DONE

```

O programa compila em **54 palavras**. Após a execução, o PC fica travado no endereço 52 (DONE).

5.5. Resultado

Os sete tons de cinza são gravados nas posições \$0100–\$0106 da RAM:

Pixel	Cálculo	Tom de cinza
0	$(1 + 0 + 0)/3$	0
1	$(1 + 1 + 1)/3$	1
2	$(0 + 0 + 6)/3$	2
3	$(0 + 51 + 51)/3$	34
4	$(0 + 3 + 12)/3$	5
5	$(112 + 8 + 83)/3$	67
6	$(41 + 73 + 153)/3$	89

Saída do ExcelCPU: 0, 1, 2, 34, 5, 67, 89

Saída do programa em C: 0, 1, 2, 34, 5, 67, 89

As duas saídas são idênticas, confirmando a correção da tradução.

6. Compilação e Execução

1. Compilar o programa desejado via terminal:

```
py compileExcelASM16.py ex1.s ROM.xlsx  
py compileExcelASM16.py ex2.s ROM.xlsx
```

2. Abrir ROM.xlsx e CPU.xlsx no Microsoft Excel.
3. Verificar que o *Iterative Calculation* está ativo (File > Options > Formulas > Enable Iterative Calculation, com Maximum Iterations = 1).
4. (Apenas para o Exemplo 1) Editar na ROM.xlsx as células correspondentes a \$0002 (largura) e \$0003 (altura).
5. Clicar no botão **Read ROM** para carregar o programa na RAM e, em seguida, no botão **Reset** para posicionar o PC em 0.
6. Executar pressionando **F9** repetidamente (cada pressionamento avança um ciclo de clock), aguardando o texto “Ready” entre as repetições. O Exemplo 1 conclui em algumas dezenas de ciclos; o Exemplo 2, por conter um laço de sete iterações, requer várias centenas.
7. Ler os resultados na RAM: \$0004 para o Exemplo 1; \$0100–\$0106 para o Exemplo 2.

7. Conclusão

A tradução dos dois algoritmos para Excel-ASM16 exigiu, além da correspondência direta entre instruções, a compreensão de particularidades da arquitetura do ExcelCPU que vão além da documentação. Os principais aprendizados foram:

- **Ausência de I/O por hardware.** A CPU não possui entrada de dados em tempo de execução. A “interação do usuário” do Exemplo 1 foi feita preparando os dados na ROM antes da execução, e a saída de texto foi substituída pela leitura direta da memória — adaptações necessárias para preservar a lógica dos programas originais.
- **O carry flag nas operações aritméticas.** As instruções ADD e SUB incorporam o carry flag. Foi necessário usar CLC antes de cada soma para evitar que resíduos de operações anteriores corrompessem o resultado.
- **Propagação em registradores altos.** Carregar um valor imediato em um registrador alto (R8–R15) e utilizá-lo na instrução seguinte não funciona de forma confiável no simulador. A solução, aplicada nos dois exercícios, foi usar registradores baixos (R0–R7) para ponteiros e divisores recém-carregados.
- **Representação de dados maiores que a palavra.** Tanto o produto de 32 bits do Exemplo 1 quanto os pixels de 24 bits do Exemplo 2 não cabem em uma palavra de 16 bits. Em ambos os casos, a solução foi dividir o dado em duas palavras (parte alta e parte baixa), tratando-as separadamente.

- **Suprir instruções ausentes.** A falta de deslocamento lógico foi contornada combinando rotação (ROR) com máscara (AND), demonstrando como operações de alto nível em C podem ser reconstruídas a partir de um conjunto de instruções reduzido.

Os dois programas finais — de 20 e 54 palavras, respectivamente — reproduzem fielmente as saídas dos originais em MIPS e em C, cumprindo o objetivo de tradução proposto.